

1 Rezolvarea problemei ESM

1.1 Prezentare generală

Problema ESM (Elemente Speciale Matematice) conține trei subpuncte separate, fiecare cerind un calcul diferit asupra aceluiași șir de numere întregi v de lungime n . Soluția prezentată tratează toate cele trei cazuri într-un singur program, folosind aceeași structură de date auxiliară.

1.2 Cazul 1: Numărarea progresiilor geometrice

Pentru $c = 1$, trebuie să numărăm tripletele de elemente consecutive (v_i, v_{i+1}, v_{i+2}) care formează o progresie geometrică, adică satisfac relația:

$$v_i \cdot v_{i+1} = v_{i+2}$$

Soluția este trivială: parcurgem toate pozițiile i de la 1 la $n - 2$ și verificăm condiția. Complexitatea este $O(n)$.

1.3 Cazul 2: Perechi de divizori pentru ultimul element

Pentru $c = 2$, trebuie să găsim valoarea maximă a expresiei $\min(\text{poz}(i), \text{poz}(j))$ pentru toate perechile de divizori (i, j) ai ultimului element v_n , unde $i \cdot j = v_n$ și $i < j$.

1.3.1 Observație cheie

Pentru fiecare valoare din șir, trebuie să cunoaștem poziția ultimei și a penultimei apariții. Folosim două tablouri:

- $\text{fr}[x]$ = ultima poziție unde apare valoarea x
- $\text{fr2}[x]$ = penultima poziție unde apare valoarea x

Aceste valori se actualizează în timpul parcurgerii:

```
for(int i = 1; i < n; i++)
{
    fr2[v[i]] = fr[v[i]];
    fr[v[i]] = i;
}
```

1.4 Cazul 3: Suma maximelor pentru toate elementele

Pentru $c = 3$, trebuie să calculăm suma valorilor $\max_{i,j=v[x]} \min(\text{poz}(i), \text{poz}(j))$ pentru fiecare poziție x de la 1 la n .

1. Inițial, fr și fr2 sunt goale (nu conțin poziții).
2. Pentru fiecare x de la 1 la n :

- Enumerăm toți divizorii i ai lui $v[x]$ (factorii proprii-zise și complementarii $j = v[x]/i$).
- Pentru fiecare pereche (i, j) validă, calculăm $\min(\text{fr}[i], \text{fr}[j])$ dacă ambele au apărut anterior.
- Reținem valoarea maximă găsită și o adăugăm la răspuns.
- Actualizăm fr2 și fr cu poziția curentă x .

1.5 Structura tablourilor de frecvență

Folosim proprietatea că pentru fiecare valoare din șir actualizăm pozițiile astfel:

$$\begin{aligned}\text{fr2}[\text{val}] &\leftarrow \text{fr}[\text{val}] && \text{(vechea poziție devine penultimă)} \\ \text{fr}[\text{val}] &\leftarrow x && \text{(poziția curentă devine ultima)}\end{aligned}$$

Această tehnică permite calculul efficient al perechilor de divizori care au apărut anterior.

1.6 Complexitate

- Cazul 1: $O(n)$
- Cazul 2: $O(\sqrt{v_n})$ pentru enumerarea divizorilor
- Cazul 3: $O(n \cdot \sqrt{v_{\max}})$ în cazul cel mai defavorabil

1.7 Implementare

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ifstream cin("esm.in");
    ofstream cout("esm.out");

    int c, n, v[100001], fr[100001] = {0}, fr2[100001] = {0};
    cin >> c >> n;

    for(int i = 1; i <= n; i++)
        cin >> v[i];

    if(c == 1)
    {
        int ans = 0;
        for(int i = 1; i + 2 <= n; i++)
            if(1LL * v[i] * v[i+1] == 1LL * v[i+2])
```

```

        ans++;
        cout << ans << '\n';
    }

    if(c == 2)
    {
        for(int i = 1; i < n; i++)
        {
            fr2[v[i]] = fr[v[i]];
            fr[v[i]] = i;
        }
        int maxi = 0;
        for(int i = 1; i * i <= v[n]; i++)
            if(v[n] % i == 0)
            {
                if(i != v[n]/i && fr[i] && fr[v[n]/i])
                    maxi = max(maxi, min(fr[i], fr[v[n]/i]));
                ;
                if(i == v[n]/i && fr2[i])
                    maxi = max(maxi, fr2[i]);
            }

        cout << maxi << '\n';
    }

    if(c == 3)
    {
        long long ans = 0;
        for(int x = 1; x <= n; x++)
        {
            int maxi = 0;
            for(int i = 1; i * i <= v[x]; i++)
                if(v[x] % i == 0)
                {
                    if(i != v[x]/i && fr[i] && fr[v[x]/i])
                        maxi = max(maxi, min(fr[i], fr[v[x]/i]));
                    ;
                    if(i == v[x]/i && fr2[i])
                        maxi = max(maxi, fr2[i]);
                }
            ans += maxi;
            fr2[v[x]] = fr[v[x]];
            fr[v[x]] = x;
        }
        cout << ans << '\n';
    }
    return 0;
}

```

1.8 Note importante de implementare

1. Conversia în `long long` în Cazul 1 este esențială pentru a evita depășirea limitelor înmulțirii numerelor întregi de 32 de biți.
2. Condiția `i != v[x]/i` verifică că divizorii sunt distincți; altfel, când $i = j$, trebuie să folosim penultima poziție din `fr2`.
3. Valoarea 0 în tablourile `fr` semnalează că elementul nu a apărut încă, deci ignorăm acele cazuri.